MICROSAR CAN Driver

Technical Reference

Vector Informatik GmbH

VTTCan

Canoe

Version 3.01.00

Authors	Holger Birke
Status	Released

1 Document Information

1.1 History

Author	Date	Version	Remarks
Holger Birke	2006-06-29	1.1	First AUTOSAR release
Holger Birke	2007-02-15	1.2	Update to Post build
Holger Birke	2007-07-25	1.3	Update to AUTOSAR specification 2.1
Holger Birke	2007-12-14	1.4	Add extended/mixed id support
Holger Birke	2008-03-03	1.5	Update to ASR 3
Holger Birke	2008-05-14	1.6	Minor Review changes
Holger Birke	2008-08-13	1.7	Update Tool screenshots for new release
Holger Birke	2008-10-24	1.8	Update Tool screenshots (ASR naming)
Holger Birke	2009-06-08	1.9	Update Core (Compiler Abstraction) + Tool screenshots for new release
Holger Birke	2009-07-20	1.10	Update Core/template + Tool screenshots for new release
Holger Birke	2011-03-15	2.00	MICROSAR 4 support added Update Core/template + Tool screenshots for new release
Holger Birke	2011-12-16	2.01	Update Core/template + Tool screenshots for new release
Holger Birke	2012-02-08	2.01.01	Fix some minor findings
Holger Birke	2012-08-10	2.02.00	MICROSAR 4.0.3 support (Core 4.03.00 – R5)
Holger Birke	2012-08-10	2.03.00	Update Core/template (Core 4.05.00 – R7)
Holger Birke	2013-10-14	2.04.00	Update Core/template (Core 4.06.00 – R8)
Holger Birke	2014-10-24	2.05.00	Update Core/template (Core 5.00.00 – R11)
Holger Birke	2015-06-30	2.05.01	ESCAN00083619 RAM check description improvement
Holger Birke	2015-07-10	2.07.00	Update Core/template (Core 5.02.00 – R13)
Holger Birke	2015-11-27	2.07.01	Update AutoSar deviation table (R14)
Holger Birke	2017-06-07	2.08.00	Add SilentMode (R19)
Holger Birke	2018-01-09	3.00.00	Add TxHwFifo, RxHwFfifo, emulate infixed driver, Accept longer DLC, multiple Baudrates, BusMirroring (R22)
Holger Birke	2021-01-21	3.01.00	Add Security Event detection (IDsM) (R26)

1.2 Reference Documents

No.	Title	Version
[1]	AUTOSAR_SWS_CAN_DRIVER.pdf	4.0.0
[2]	AUTOSAR_BasicSoftwareModules.pdf	V1.0.0
[3]	AUTOSAR_SWS BSW Scheduler	V1.1.0
[4]	AUTOSAR_SWS_CAN_Interface.pdf	5.0.0
[5]	AN-ISC-8-1118 MICROSAR BSW Compatibility Check	V1.0.0
[6]	ISO DIS 11898-1	2015

1.3 Scope of the Document

This document describes the functionality, API and configuration of the MICROSAR CAN Driver as specified in [1]. The CAN Driver is a hardware abstraction layer with a standardized interface to the CAN Interface layer.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector’s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Document Information 2
1.1 History 2
1.2 Reference Documents 3
1.3 Scope of the Document 3

2 Hardware Overview 7
2.1 Hardware Software Interface (HSI) 7

3 Introduction 8
3.1 Architecture Overview 9

4 Functional Description 11
4.1 Features 11
4.2 Initialization 14
4.3 Communication 14
4.4 States / Modes 16
4.5 Re-Initialization 18
4.6 CAN Interrupt Locking 18
4.7 Main Functions 18
4.8 Error Handling 18
4.9 Hardware Specific 22

5 Integration 23
5.1 Scope of Delivery 23
5.2 Include Structure 24
5.3 Critical Sections 24
5.4 Compiler Abstraction and Memory Mapping 26
5.5 Hardware Specific Hints 28

6 API Description 29
6.1 Can_InitMemory 29
6.2 Can_Init 29
6.3 Can_ChangeBaudrate 30
6.4 Can_CheckBaudrate 30
6.5 Can_SetBaudrate 31
6.6 Can_GetVersionInfo 32
6.7 Can_GetStatus 32
6.8 Can_GetControllerMode 33
6.9 Can_GetControllerErrorState 34
6.10 Can_GetControllerTxErrorCounter 34

6.11 Can_GetControllerRxErrorCounter 35
6.12 Can_SetControllerMode 36
6.13 Can_Write 36
6.14 Can_CancelTx 37
6.15 Can_SetMirrorMode 37
6.16 Can_SetSilentMode 38
6.17 Can_CheckWakeup 39
6.18 Can_DisableControllerInterrupts 39
6.19 Can_EnableControllerInterrupts 40
6.20 Can_MainFunction_Write 40
6.21 Can_MainFunction_Read 41
6.22 Can_MainFunction_BusOff 41
6.23 Can_MainFunction_Wakeup 42
6.24 Can_MainFunction_Mode 43
6.25 Can_RamCheckExecute 43
6.26 Can_RamCheckEnableMailbox 44
6.27 Can_RamCheckEnableController 44
6.28 CanSetHighVoltageMode 45
6.29 Can_RxIndication 45
6.30 Can_TxConfirmation 46
6.31 Can_ErrIndication 47
6.32 CanInterruptRx 47
6.33 CanInterruptTx 48
6.34 CanInterruptErr 48
6.35 CanInterruptWakeup 49
6.36 Appl_GenericPrecopy 49
6.37 Appl_GenericConfirmation 50
6.38 Appl_GenericConfirmation 51
6.39 Appl_GenericPreTransmit 52
6.40 ApplCanTimerStart 52
6.41 ApplCanTimerLoop 53
6.42 ApplCanTimerEnd 54
6.43 Appl_CanOverrun 54
6.44 Appl_CanFullCanOverrun 55
6.45 Services used by CAN 55

7 Configuration 57
7.1 Pre-Compile Parameters 57
7.2 Link-Time Parameters 58
7.3 Post-Build Parameters 58
7.4 Configuration with da DaVinci Configurator 59

8 Glossary and Abbreviations 60
8.1 Glossary 60
8.2 Abbreviations 60

9 Contact 61

Illustrations

Figure 3-2 AUTOSAR architecture 9
Figure 3-3 Interfaces to adjacent modules of the CAN 10
Figure 5-1 Include Structure (AUTOSAR) 24

Tables

Table 2-1 Supported Hardware Overview 7
Table 4-1 Supported features 14
Table 4-2 Hardware mailbox layout 15
Table 4-3 Errors reported to DET 19
Table 4-4 API from which the Errors are reported 20
Table 4-6 Hardware Loop Check 21
Table 5-1 Static files 23
Table 5-2 Generated files 23
Table 5-3 Critical Section Codes 26
Table 5-4 Compiler abstraction and memory mapping 27
Table 6-1 Services used by the CAN 56
Table 9-1 Glossary 60
Table 9-2 Abbreviations 60

2 Hardware Overview

The following table summarizes information about the CAN Driver. It gives you detailed information about the derivatives and compilers. As very important information the documentations of the hardware manufacturers are listed. The CAN Driver is based upon these documents in the given version.

Derivative	Compiler	Hardware Manufacturer Document	Version
VttCan	Visual-C	Vector Informatik GmbH	-

Table 2-1 Supported Hardware Overview

Derivative: This can be a single information or a list of derivatives, the CAN Driver can be used on.
Compiler: List of Compilers the CAN Driver is working with
Hardware Manufacturer Document Name: List of hardware documentation the CAN Driver is based on.
Version: To be able to reference to this hardware documentation its version is very important.

2.1 Hardware Software Interface (HSI)

The used CAN cell is accessed by using the CANoe VIA communication API (refer to VIA documentation). VTT OS provide all necessary timing and Interrupt controls.

3 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module CAN as specified in [1].

Since each hardware platform has its own behavior based on the CAN specifications, the main goal of the CAN Driver is to give a standardized interface to support communication over the CAN bus for each platform in the same way. The CAN Driver works closely together with the higher layer CAN Interface.

Supported AUTOSAR Release*:	4	
Supported Configuration Variants: **(Supported AUTOSAR Standard Conform Features)**	Pre-Compile Post-Build Loadable Post-Build Selectable (MICROSAR Identity Manager)	
Vendor ID:	CAN_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	CAN_MODULE_ID	80 decimal (according to ref. [2])
AR Version:	CAN_AR_RELEASE_MAJOR_VERSION CAN_AR_RELEASE_MINOR_VERSION CAN_AR_RELEASE_REVISION_VERSION	AUTOSAR Release version (BCD coded)
SW Version:	CAN_SW_MAJOR_VERSION CAN_SW_MINOR_VERSION CAN_SW_PATCH_VERSION	MICROSAR Can module version (BCD coded)

* For the precise AUTOSAR Release 4.x please see the release specific documentation.

3.1 Architecture Overview

The following figure shows where the CAN is located in the AUTOSAR architecture.

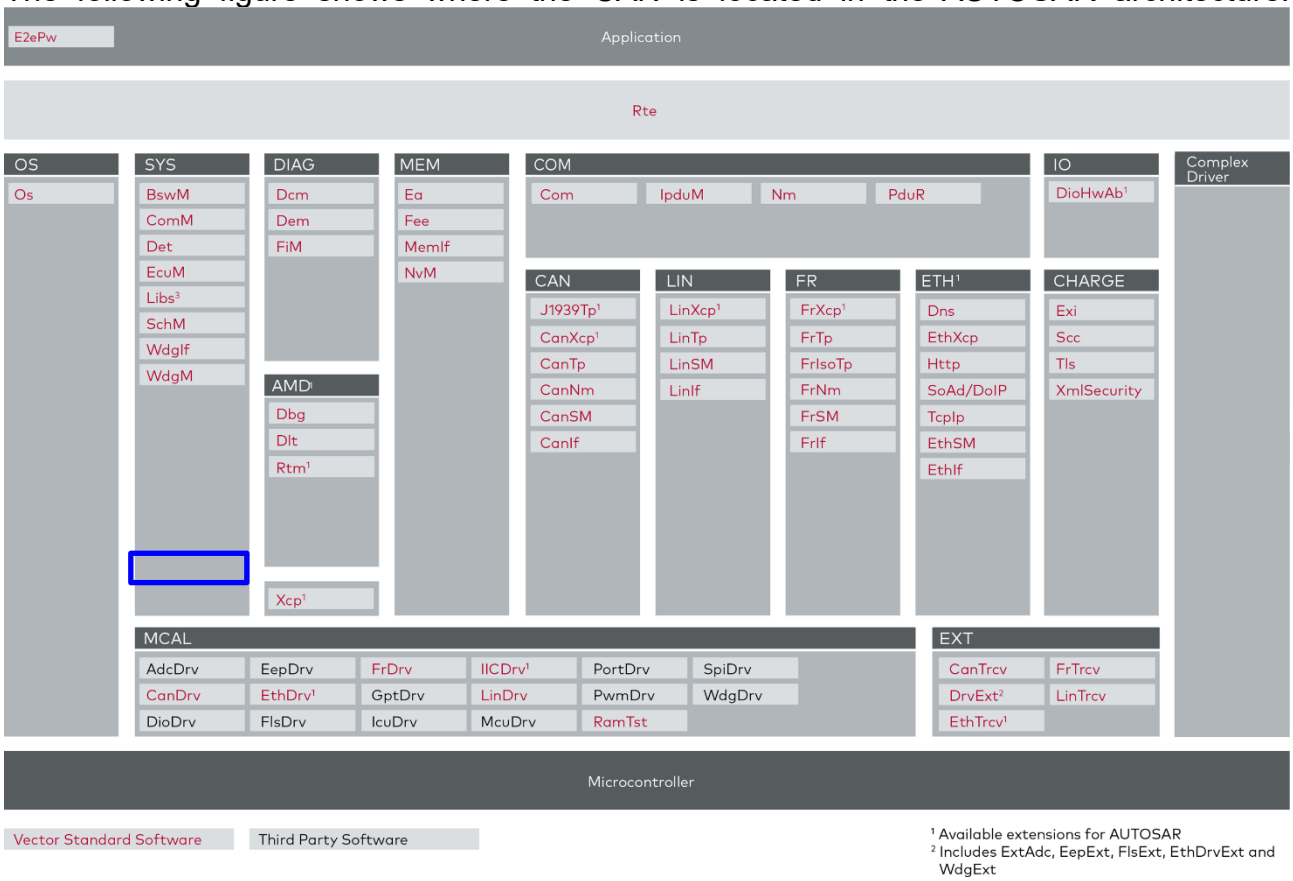


Figure 3-1 AUTOSAR 3.x Architecture Overview

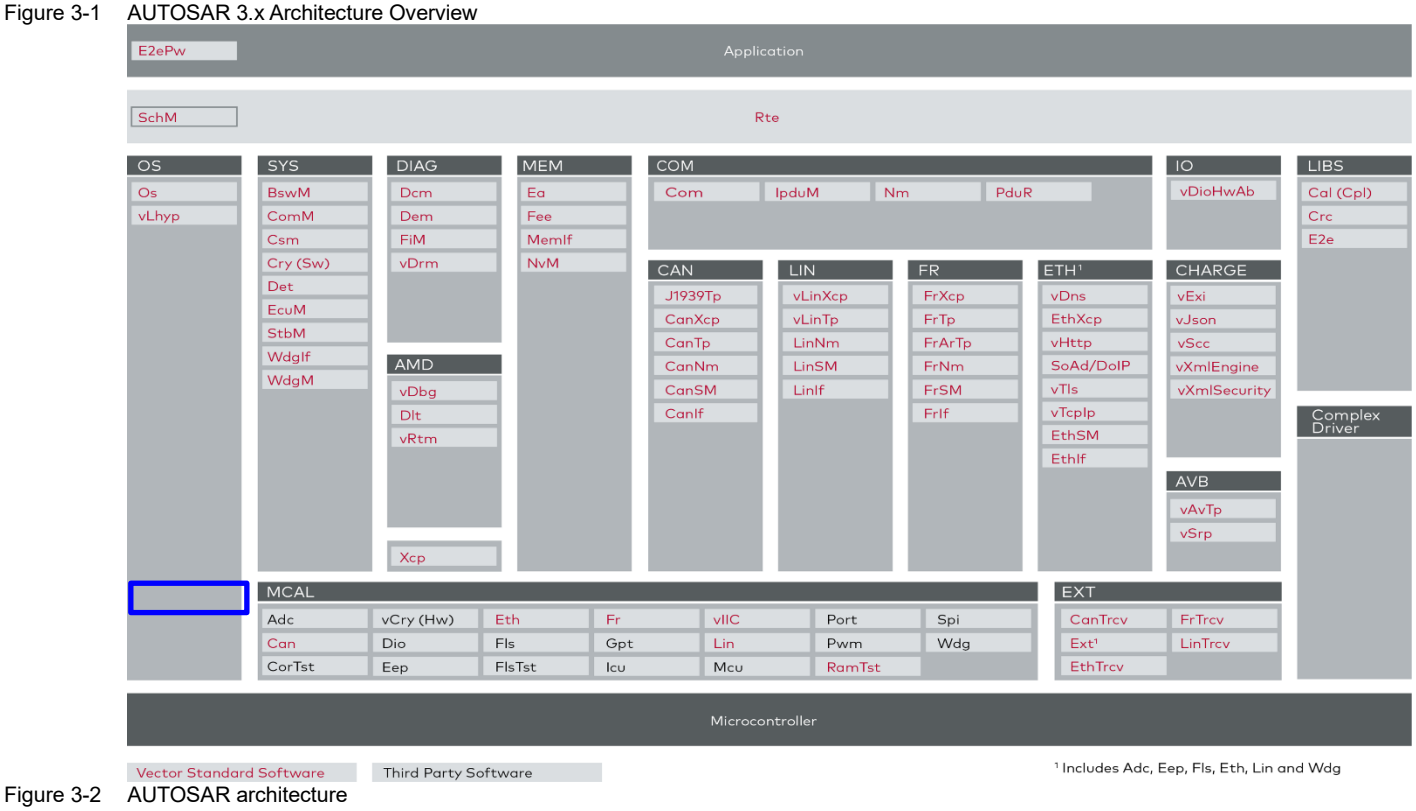


Figure 3-2 AUTOSAR architecture

The next figure shows the interfaces to adjacent modules of the CAN. These interfaces are described in chapter 6.

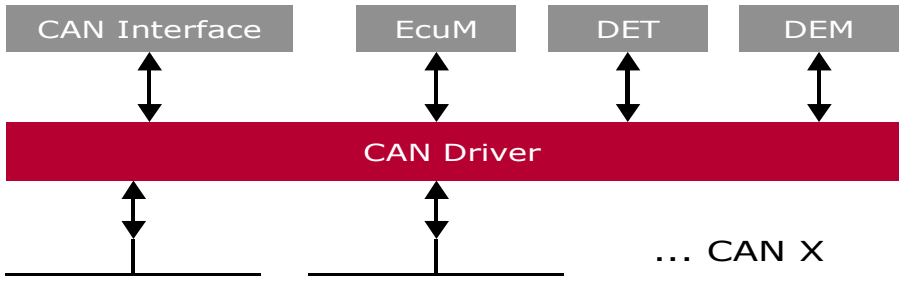


Figure 3-3 Interfaces to adjacent modules of the CAN

4 Functional Description

4.1 Features

The features listed in this chapter cover the complete functionality specified in [1].

The "supported" and "not supported" features are presented in the following table.

Feature Naming	Short Description	supported
Initialization		
PowerOnInit	General driver initialization function Can_Init()	■
DeInit	Support uninitialization routine (since ASR 4.4.0)	□
InitMemory	Support initialization of memory at power on.	■
Communication		
Transmission	Transmitting CAN frames.	■
Transmit confirmation	Callback for successful Transmission.	■
Reception	Receiving CAN frames.	■
Receive indication	Callback for receiving frame.	■
Controller Modes		
Sleep	Controller support `SLEEP` mode (power saving).	■
Wakeup over CAN	Controller support `WAKEUP` over CAN. Limitation: Can_CheckWakeup() is not encapsulated by preprocessor switch (since ASR 4.2.0) (Vtt only emulate the wakeup by parsing RX signal)	■
Stop	Controller support `STOP` mode (passive to CAN bus).	■
Bus Off detection	Callback for `BUSOFF` event.	■
Silent Mode	Support Silent Mode where the controller only listen passive.	■
Wakeup over ICU	Support wakeup over ICU (since ASR 4.2.0).	□
MirrorMode / BusMirroring	Support message mirroring where the controller support generic confirmation function for mirroring and support an API to activate and deactivate this.	■
Polling Modes		
Tx Confirmation	Support polling mode for Transmit confirmation.	■
Rx Reception	Support polling mode for Reception.	■
Wakeup	Support polling mode for `WAKEUP` event.	■
Bus Off	Support polling mode for `BUSOFF` event.	■
Mode	Support polling mode for mode transition.	■
Mailbox objects		
Tx BasicCAN	Standard mailbox to send CAN frames (Used by CAN Interface data queue).	■

Tx Hardware FIFO	Using a FIFO mechanism for Tx BasicCAN mailbox (size can be configured)	■
Multiplexed Tx	Using 3 mailboxes for Tx BasicCAN mailbox (external priority inversion avoided).	■
Tx FullCAN	Separate mailbox for special Tx message used.	■
Maximum amount	Available amount of mailboxes.	250
Rx FullCAN	Separate mailbox for special Rx message used.	■
Maximum amount	Available amount of mailboxes.	250
Rx BasicCAN	Standard mailbox to receive CAN frame (as a configurable sized FIFO).	■
Maximum amount	Available amount of BasicCAN objects.	250
CanHardwareObject SymbolicName values	Limitation: “Symbolic Name Values” may change their values after precompile phase so do not use it for Link-time or Post-build variants. (since ASR 4.0.0)	■
Others		
DET	Support Development Error Detection (error notification).	■
Version API	API to read out component version.	■
Maximum supported controllers	Maximum amount of supported controllers (hardware channels).	16
Cancellation of Tx objects	Support of Tx Cancellation (out of hardware). Avoid internal priority inversion. VttCan only emulates this feature but does not really cancel the messages.	■
Identical ID cancellation	Tx Cancellation also for identical IDs.	■
Standard ID types	Standard Identifier supported (Tx and Rx).	■
Extended ID types	Extended Identifier supported (Tx and Rx).	■
Mixed ID types	Standard and Extended Identifier supported (Tx and Rx).	■
CAN FD Mode1	FD frames with baudrate switch (BRS; Tx and Rx).	■
CAN FD Mode2	FD frames up to 64 data bytes (FULL support; Tx and Rx).	■
Hardware Loop Check (Timeout monitoring)	To avoid possible endless loops (occur by hardware issue). VttCan only emulates this feature all loops will be called limited times.	■
Multiple CAN driver	API infixed CAN driver support	■
Pretended Networking	Support pretended networking (since ASR 4.2.0)	☐
Individual Polling	Support individual polling mode (selectable for each mailbox separate). Limitation: AutoSar feature multiple cyclic functions with different time periods are not supported. The polling take part in the same main function calls (since ASR 4.2.0).	■*
TriggerTransmit	Support trigger transmit (since ASR 4.2.0)	☐
EcuC partition map	Support EcuC partition mapping (since ASR 4.4.0).	☐

Security Event Reporting	Support security event reporting (since ASR 20-11)	■
AutoSar extensions		
Multiple Rx Basic CAN	Support Multiple Rx BasicCAN objects. This gives the possibility to use multiple filters and optimize acceptance Filtering as well as avoid OVERRUN.	■*
Multiple Tx Basic CAN	Support Multiple Tx BasicCAN objects. Used to send different Tx groups over separate mailboxes with different buffering behavior (see CanInterface). (“Cancellation of Tx objects” is not possible with this feature activated)	■*
Rx Queue	Support Rx Queue. This give the possibility to buffer received data in interrupt context but handle it asynchronous in polling task.	■*
Secure Rx Buffer used	Special hardware buffer used to temporary save received data.	□
Hardware Loop Check by application	“Hardware Loop Check” can be defined to be done by application (special API available)	■
Configurable “Nested CAN interrupts”	Nested CAN interrupts allowed, and can be also switched to none-nested. Nested means that higher prior interrupts may appear in the already active CAN interrupt context.	□
Support Mixed ID	Force CAN Driver to handle mixed ID (standard and extended ID) at pre-compile-time to expand the ID type later on.	■
Optimize for one controller	Activate this for 1 controller systems when you never will expand to multi-controller. So that the CAN Driver works more efficient	■
Size of Hw HandleType	Support 8bit or 16bit Hardware Handles depend on hardware usage.	■
Generic PreCopy	Support a callback function for receiving any CAN message (following callbacks could be suppressed) Limitation: AutoSar feature name is LPDU callback function this is not fully supported because the API signature is different (since ASR 4.2.0)	■
Generic Confirmation	Support a callback function for successful transmission of any CAN message (following callbacks could be suppressed)	■
Get Hardware Status	Support a API to get hardware status information (see Can_GetStatus(), Can_GetControllerStatus(), Can_GetControllerMode(), CanGetControllerRxErrorCounter(), CanGetControllerTxErrorCounter())	■
Interrupt Category selection	Support Category 1 or Category 2 Interrupt Service Routines for OS	■
Overrun Notification	Support DET or application notification cause by OVERRUN (OVERWRITE) of an Rx message (BasicCAN and FullCAN) Limitation: overrun will be only notified when DET is activated (since ASR 4.4.0 the overrun will be also notified without DET)	■
RAM Check	Support CAN mailbox RAM check	■
Extended RAM Check	Support extended RAM check. Handling of individual deactivated mailboxes and controllers.	■

Generic PreTransmit	Support a callback function with pointer to Data, right before this data will be written in hardware mailbox buffer to send. (Use this to change data right before transmission)	■

Table 4-1 Supported features

■ Feature is supported

☐ Feature is not supported

* HighEnd Licence only

4.2 Initialization

`Can_Init()` has to be called to initialize the CAN Driver at power on and initialize the CAN controllers.

After this call the controller stays in `STOP` mode until the CAN Interface changes to `START`.

`Can_InitMemory()` is an additional service function to reinitialize the memory to bring the driver back to a pre-power-on state (not initialized). Afterwards `Can_Init()` has to be called again. It is recommended to use this function before calling `Can_Init()` to secure that no startup-code specific pre-initialized variables affect the driver startup behavior.

4.3 Communication

`Can_Write()` is used to send a message over the mailbox object given as "Hth". The data, DLC and ID is copied into the hardware mailbox object and a send request is set. After sending the message the CAN Interface `CanIf_TxConfirmation()` function is called. Right before the data is copied in mailbox buffer the ID, DLC and data may be changed by `Appl_GenericPreTransmit()` callback.

When “Generic Confirmation“ is activated the callback `Appl_GenericConfirmation`() will be called before `CanIf_TxConfirmation()` and the call to this can be suppressed by `Appl_GenericConfirmation`() return value.

For Tx messages the ID will be copied. (Exception: feature “Dynamic FullCAN Tx ID” is deactivated, then the FullCAN Tx messages will be only set while initialization)

If the mailbox is currently sending the status busy will be returned. Then the message may be queued in the CAN Interface (if feature is active).

If cancellation in hardware is supported the lowest priority ID inside currently sending object is canceled, and therefore re-queued in the CAN Interface.

`Appl_GenericPreCopy`() (if activated) is called and depend on return value also `CanIf_RxIndication()` as a CAN Interface callback, is called when a message is received. The receive information like ID, DLC and data are given as parameter.

When Rx Queue is activated the received messages (polling or interrupt context) will be queued (same queue over all channels). The Rx Queue will be read by calling `Can_Mainfunction_Read` () and the Rx Indication (like `CanIf_RxIndication`()) will be

called out of this context. Rx Queue is used for Interrupt systems to keep interrupt latency time short.

4.3.1 Mailbox Layout

The generation tool supports a flexible allocation of message buffers. In the following tables the possible mailbox layout is shown (the range for each mailbox types depend on the used mailboxes).

Object type	No. of Objects	Comment
Tx FullCAN	0-250	These objects are used to send a certain message. The user must define statically (Generation Tool) which CAN messages are located in such Tx FullCAN objects. The Generation Tool distributes the messages to the FullCAN objects.
Tx BasicCAN	Sum($Y_{(x)}$)	These objects are used to send several messages. If the transmit message object is busy, the transmit request is stored in a CAN Interface queue (if activated). X = amount of TxBasicCANs, Y = 1…n depend on 'Multiplexed Tx' and 'Tx Hardware FIFO' feature (individual for each TxBasicCAN)
Rx FullCAN	0-250	These objects are used to receive specific CAN messages. The user defines statically (Generation Tool) that a CAN message should be received in a FullCAN message object. The Generation Tool distributes the message to the FullCAN objects.
Rx BasicCAN	0-250	All other CAN messages are received via the BasicCAN message object. Each BasicCAN message object consists of ??? message buffers (??? FIFO) and (??? amount) filter. If using "Multiple BasicCAN" feature the amount of used BasicCAN can be configured (use also this feature to select no BasicCAN)
Unused	0-250	These objects are not used. It depends on the configuration of receive and transmit objects if unused objects are available.

Table 4-2 Hardware mailbox layout

The "CanObjectId" (ECUc parameter) numbering is done in following order: FullCAN Tx, BasicCAN Tx, Unused, FullCAN Rx, BasicCAN Rx (like shown above). "CanObjectId's" for next controller begin at end of last controller. Gaps in "CanObjectId" for unused mailboxes may occur.

4.3.2 Mailbox Processing Order

The hardware mailbox will be processed in following order:

Object Type	Order / priority to send or receive
Tx FullCAN	FIFO (all mailboxes will be send out over one common queue)
Tx BasicCAN	FIFO (all mailboxes will be send out over one common queue)
Rx FullCAN	FIFO (all mailboxes will be received over one common queue)
Rx BasicCAN	FIFO (all mailboxes will be received over one common queue)

In Case of Polling and Interrupt, FullCANs will be processed before BasicCANs.

The order between Rx and Tx mailboxes depends on the call order of the polling tasks or the interrupt context and cannot be guaranteed.

The Rx Queue will work like a FIFO filled with the method mentioned above.

4.3.3 Acceptance Filter for BasicCAN

The hardware acceptance filter of the BasicCAN object will be emulated in software. The filter in CANoe may remain full open and the CAN driver reject the unused messages according to the filter settings in the CAN driver. One receive mailbox will be used for each filter (standard or extended ID). For mixed ID used case please configure 2 Filters, one as standard and one as extended.

If no message should be received, select the “Multiple Basic CAN” feature and set the amount to 0. Otherwise the filter should be set to “close”. Use feature “Rx BasicCAN Support” to deactivate unused code (for optimization).

4.3.4 Remote Frames

Remote Frames rejected in Hardware	Remote Frames rejected in Software
no	**no**

Remote Frames cannot be filtered for this platform they will be received like normal message.

4.4 States / Modes

You can change the CAN Cell mode via Can_SetControllerMode(). The last requested transition will be executed. The upper layer should take care about valid transitions.

Following mode changes are supported:

```
CAN_T_START
CAN_T_STOP
CAN_T_SLEEP (only emulated)
CAN_T_WAKEUP
```

Notification of mode change may occur asynchronous by notification `CanIf_ControllerModeIndication()`

4.4.1 Start Mode (Normal Running Mode)

This is the mode where communication is possible. This mode has to be set after initialization because the controller is first in `STOP`.

4.4.2 Stop Mode

The stop mode is an emulated mode for this platform and do not really disconnect from CAN bus. Due to that Acknowledge will be send for incoming messages.

4.4.3 Sleep Mode

The Sleep mode and the Wakeup event are only emulated. This means that a Acknowledge will be send, the controller just remember his state. A received message will cause a wakeup-event and is also received (notification).

`EcuM_CheckWakeup()` is called in case a `WAKEUP` over CAN event occurred.

`Can_Cbk_CheckWakeup()` can be called to check `WAKEUP` state.

`CAN_T_WAKEUP` has to be set with `Can_SetControllerMode()` to leave this mode.

If wakeup support is deactivated by configuration, the controller went into `STOP` mode instead of `SLEEP`.

4.4.4 Bus Off

`CanIf_ControllerBusOff()` is called when the controller detects a `BUSOFF` event. The mode is automatically changed to `STOP`. The upper layers have to care about returning to normal running mode by calling `START`.

4.4.5 Silent Mode

Support API (`Can_SetSilentMode()`) to switch into ‘Silent Mode’ where the controller does not take part on CAN bus communication (no ACK) but can listen for messages. Refer to ISO 11898 bus monitoring.
Change in ‘Silent Mode’ and back again to normal operating mode is only possible in `STOP`. To get RX notifications in ‘Silent Mode’ it is necessary to switch to `START` mode.
In ‘Silent Mode’ it is possible to change the baud rate and switch to `START` mode, used to scan baud rates by detect reception.



Caution
Don’t use any other API than Can_SetSilentMode(x, CAN_SILENT_INACTIVE) inside activated “Silent Mode” except Can_SetControllerMode(`START` or `STOP`) and Can_ChangeBaudrate() and Can_SetBaudrate().

If you use CAN FD baud rates to be scanned, use a FD message for detection.

Canoe as an emulation does not support a internal rerouting so if there is no other node that acknowledge the bus no message will be received.

Rx error counter will be increased until passive for each not acknowledged message.

4.5 Re-Initialization

A call to `Can_ChangeBaudrate() or Can_SetBaudrate()` cause a re-initialization of a dedicated CAN Controller. Pending messages may be processed before the transition will be finished. A re-initialization is only possible out of `STOP` and does not change to another mode.

After re-initialization all CAN communication relevant registers are set to initial conditions.

4.6 CAN Interrupt Locking

`Can_DisableControllerInterrupts()` and `Can_EnableControllerInterrupts()` are used to disable and enable the controller specific Interrupt, Rx, Tx, `WAKEUP` and `BUSOFF` (/ Status) together. These functions can be called nested.

4.7 Main Functions

`Can_MainFunction_Write()`, `Can_MainFunction_Read()`, `Can_MainFunction_BusOff()` and `Can_MainFunction_Wakeup()` called by upper layer to poll the events if the specific polling mode is activated. Otherwise these functions return without action and the events will be handled in interrupt context.

When individual polling is activated only mailboxes that are configured as to be polled will be polled in the main functions "`Can_MainFunction_Write()`" and "`Can_MainFunction_Read()`" all others handled in interrupt context.

When Rx Queue is activated the queue is filled in interrupt or polling context like configured. But the processing (indications) will be done in "`Can_MainFunction_Read()`" context.

Can_MainFunction_Mode() called by upper layer to poll asynchronous mode transition notifications.

4.8 Error Handling

4.8.1 Development Error Reporting

Development errors are reported to DET using the service `Det_ReportError()`, if the pre-compile parameter CAN_DEV_ERROR_DETECT == STD_ON.

The tables below, shows the API ID and Error ID given as parameter for calling the DET.

Instance ID is always 0 because no multiple instances are supported.

Errors reported to DET:	
Error ID	**Short Description**
CAN_E_PARAM_POINTER	API gets an illegal pointer as parameter.
CAN_E_PARAM_HANDLE	API get an illegal handle as parameter
CAN_E_PARAM_DLC	API get an illegal DLC as parameter
CAN_E_PARAM_CONTROLLER	API get an illegal controller as parameter

Errors reported to DET:	
Error ID	**Short Description**
CAN_E_UNINIT	Driver API is used but not initialized
CAN_E_TRANSITION	Transition for mode change is illegal
CAN_E_DATALOST (value: 0x07, AutoSar extension)	Rx `OVERRUN` (`OVERWRITE`) detected
CAN_E_PARAM_BAUDRATE (value: 0x08, AutoSar extension)	Selected baudrate is not valid
CAN_E_RXQUEUE (value: 0x10, AutoSar extension)	Rx Queue `OVERRUN` (last received message is lost, and will not be received → increase queue size)
CAN_E_TIMEOUT_DET (value: 0x11, AutoSar extension)	Hardware Loop timeout detected.

Table 4-3 Errors reported to DET

API from which the errors reported to DET:	
API ID	**Functions using that ID**
CAN_INIT_ID	Can_Init()
CAN_INITCTR_ID	CanHL_ReInit()
CAN_SETCTR_ID	Can_SetControllerMode()
CAN_DIINT_ID	Can_DisableControllerInterrupts()
CAN_ENINT_ID	Can_EnableControllerInterrupts()
CAN_WRITE_ID	Can_Write(), Can_CancelTx()
CAN_VERSION_ID	Can_GetVersionInfo()
CAN_MAINFCT_WRITE_ID	Can_MainFunction_Write()
CAN_MAINFCT_READ_ID	Can_MainFunction_Read()
CAN_MAINFCT_BO_ID	Can_MainFunction_BusOff()
CAN_MAINFCT_WU_ID	Can_MainFunction_Wakeup()
CAN_MAINFCT_MODE_ID	Can_MainFunction_Mode()
CAN_CKWAKEUP_ID	Can_CheckWakeup()
CAN_CHANGE_BR_ID	Can_ChangeBaudrate()
CAN_CHECK_BR_ID	Can_CheckBaudrate()
CAN_SET_BR_ID	Can_SetBaudrate()
CAN_GETCTR_STATE_ID	Can_GetControllerErrorState()
CAN_GETCTR_MODE_ID	Can_GetControllerMode()
CAN_GETCTR_RXCNT_ID	Can_GetControllerRxErrorCounter()
CAN _GETCTR_TXCNT_ID	Can_GetControllerTxErrorCounter()
CAN_RXINDI_ID (AUTOSAR extension)	CanBasicCanMsgReceived(), CanFullCanMsgReceived()

CAN_TXCNF_ID (AUTOSAR extension)	CanHL_TxConfirmation()
CAN_CTRWAKEUP_ID (AUTOSAR extension)	CanHL_WakeUpHandling()
CAN_CTRBUSOFF_ID (AUTOSAR extension)	CanHL_ErrorHandling()
CAN_HW_ACCESS_ID (AUTOSAR extension)	Used for some none AUTOSAR API: Can_GetStatus(), Can_SetMirrorMode(), Can_SetSilentMode(), Can_RamCheckExecute(), Can_RamCheckEnableController() And CAN driver internal functionality: ApplCanTimerLoop()

Table 4-4 API from which the Errors are reported

4.8.1.1 Parameter Checking

AUTOSAR requires that API functions check the validity of their parameters (Refer to [1]). These checks are for development error reporting and can be enabled and disabled separately. Refer to the configuration chapter where the enabling/disabling of the checks is described. Enabling/disabling of single checks is an addition to the AUTOSAR standard which requires enable/disable the complete parameter checking via the parameter `CAN_DEV_ERROR_DETECT`.

4.8.1.2 Overrun/Overwrite Notification

This platform perform overwrite detection for FullCAN and BasicCAN receive mailbox.

As AUTOSAR extension the `OVERRUN` detection may be activated by configuration tool. The notification can be configured to call an DET or application call `Appl_CanOverrun`() or `Appl_CanFullCanOverrun`().

4.8.2 Hardware Loop Check / Timeout Monitoring

The feature “Hardware Loop Check” is used to break endless loops caused by software or hardware issue. This feature is configurable see Chapter 7 and also Timeout Duration description.

Synchronous part of mode transitions will be also limited by this timeout mechanism, which is no issue but a timing limit. (following asynchronous part of mode transition is handled without HardwareLoopCheck)

The Hardware Loop Check will be handled by CAN driver internal except when setting “Hardware Loop Check by Application” is activated.

Nevertheless, refer to “short description” below there may be activities that should be initiated by the application (e.g. reset of CAN Controller or special mode transitions). In this case the “Hardware Loop Check by Application” is recommended to be used to handle the explicit loop like described.

4.8.2.1 Critical Loops

A loop exception has to be handled by application like described below.

Loop Name / source	Short Description
None	

4.8.2.2 Uncritical Loops

No additional application handling needed after loop break.

Loop Name / source	Short Description
kCanLoopRx	- Loop to detect inconsistent data while message reception. - Call context: Can_MainfunctionRead() or Rx Interrupt. - Expected duration: 1 - Action after timeout: None. (Only simulation of a loop)

Driver handle the mode transition in asynchronous way afterwards, so no additional handling is necessary.

Loop Name / source	Short Description
kCanLoopMode	Used for short time mode transition blocking (short synchronous timeout). Use for mode changes START, STOP, SLEEP and WAKEUP. No issue when timeout occurs.

Table 4-5 Hardware Loop Check

4.8.3 CAN RAM Check

The CAN Driver supports a check of the CAN Controller's mailbox data registers (ID, DLC, data). The CAN Controller RAM check is called internally every time a power on is executed within Initialization, or a CAN bus `WAKEUP` event happen. The CAN Driver verifies that no used mailboxes are corrupt. A mailbox is considered corrupt if a predefined pattern is written to the appropriate mailbox registers and the read operation does not return the expected pattern. If a corrupt mailbox is found the function `Appl_CanCorruptMailbox()` is called. This function tells the application which mailbox is corrupt.

After the check of all mailboxes the CAN Driver calls the call back function `Appl_CanRamCheckFailed()` if at least one corrupt mailbox was found. The application must decide if the CAN Driver disables communication or not by means of the call back function's return value. If the application has decided to disable the communication there is no possibility to enable the communication again until the next call to Can_Init() or a `WAKEUP` event occurs.

The CAN RAM check functionality itself can be activated via Generation Tool.

4.8.4 Extended RAM Check

The CAN Driver supports a check for all accessible CAN Controller's control registers and mailbox registers. The extended RAM check will be executed while power on initialization and by direct call. Mailboxes will be deactivated when pattern check or configured values are corrupt. The CAN Controller will be deactivated when one mailbox is corrupt or the controller's registers failed the pattern check or configured values are corrupt. Mailbox and controller stay deactivated until they are explicit activated again.

API to execute extended RAM check:

`Can_RamCheckExecute()`

Callouts to notify corrupt mailboxes or controllers:

`CanIf_RamCheckCorruptController(), CanIf_RamCheckCorruptMailbox()`

API to activate the mailbox or controller again:

`Can_RamCheckEnableMailbox(),Can_RamCheckEnableController()`

(See also API description)

The CAN RAM check functionality itself can be activated via Generation Tool.

4.8.5 Security Event Reporting

The CAN driver supports detection and reporting of security events observed by the CAN controller. The reporting will be executed during the driver error handling, either in interrupt, or polling context. The associated context data will be reported to the CanIf when a security event is detected.

The supported security events are listed in Table 4-6

Security Event	Short Description
Error state passive	Reported when the CAN driver detects a transition to error state passive
Error state bus off	Reported when the CAN driver detects a bus off event

Table 4-6 Security Events

4.9 Hardware Specific

CANoeEmu CAN driver is an emulation driver so it may react in some cases different form a real hardware. In special see STOP mode.



Info
No OS or interrupt handler shall enable the global interrupt before entering the CAN interrupt service routines!

5 Integration

This chapter gives necessary information for the integration of the MICROSAR CAN into an application environment of an ECU.

5.1 Scope of Delivery

The delivery of the CAN contains the files, which are described in the chapter's 5.1.1 and 5.1.2:

Dependent on library or source code delivery the marked (+) files may not be delivered.

5.1.1 Static Files

File Name	Description
Can_Local.h	This is an internal header file which should not be included outside this module
Can.c	This is the source file of the CAN. It contains the implementation of CAN module functionality.
Can.h	This is the header file of the CAN module (include API declaration)
Can_Irq.c	This is the interrupt declaration and callout file (supports interrupt configuration as link time settings)

Table 5-1 Static files

5.1.2 Dynamic Files

The dynamic files are generated by the configuration tool.

File Name	Description
Can_Cfg.h	Generated header file, contains some type, prototype and pre-compile settings
Can_Lcfg.c	Generated file contains link-time settings.
Can_PBcfg.c	Generated file contains post-build settings.
Can_DrvGeneralTypes.h	Generated file contains CAN driver part of Can_GeneralTypes.h (supported by Integrator)

Table 5-2 Generated files

5.2 Include Structure

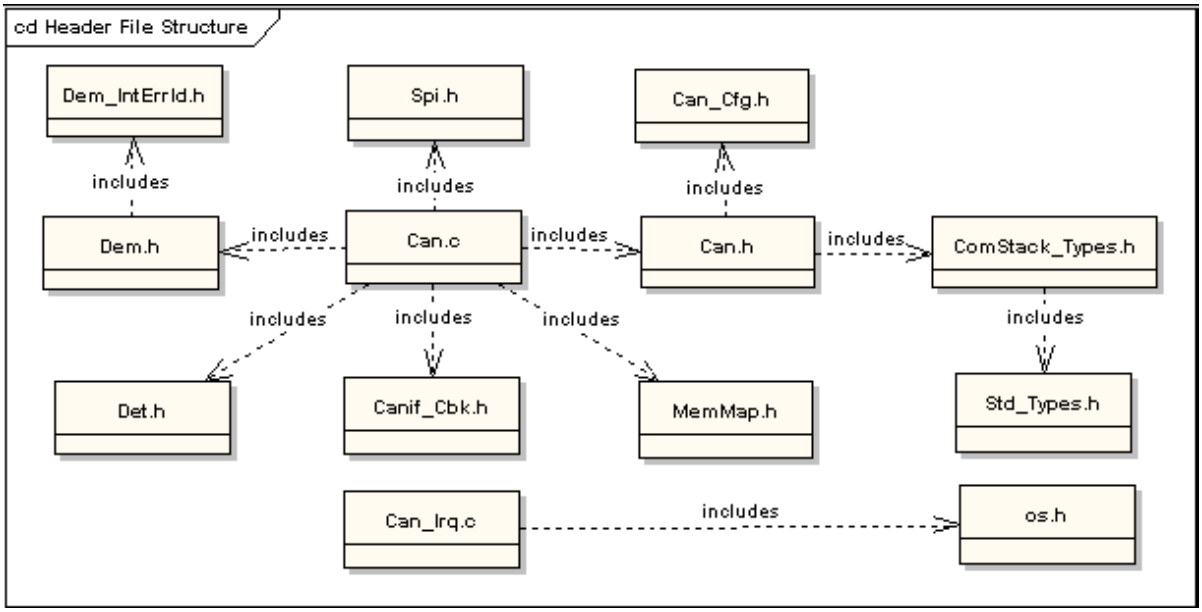


Figure 5-1 Include Structure (AUTOSAR)

Deviation from AUTOSAR Specification:

- Additionally the EcuM_Cbk.h is included by Can_Cfg.h (needed for wakeup notification API).
- ComStack_Types.h included by Can_Cfg.h, because the specified types have to be known in generated data as well.
- Os.h will be included by Can_Cfg.h because of used data types
- Spi.h is not yet used.
- Can_GeneralTypes.h will be included by Can_Cfg.h not by Can.h direct.
- Dem.h is not used

5.3 Critical Sections

The AUTOSAR standard provides with the BSW Scheduler a BSW module, which handles entering and leaving critical sections.

For more information about the BSW Scheduler please refer to [3]. When the BSW Scheduler is used the CAN Driver provides critical section codes that have to be mapped by the BSW Scheduler to following mechanism:

Critical Section Define	Description
CAN_EXCLUSIVE_AREA_1	Using inside Can_DisableControllerInterrupts() and Can_EnableControllerInterrupts() to avoid nested call of these APIs. > Duration is short > No API call of other BSW inside. > Disable global interrupts – or – Empty in case Can_Disable/EnableControllerInterrupts() are called within context with lower or equal priority than CAN interrupt and Can_DisableControllerInterrupts(), Can_EnableControllerInterrupts() are not called nested (each other and themselves).
CAN_EXCLUSIVE_AREA_2	Using inside Can_Write() to secure software states of transmit objects. (Only called when no Vector CAN Interface is used.) > Duration is medium > No API call of other BSW inside. > Disable global interrupts - or - Disable CAN interrupts and do not call function Can_Write() reentrant.
CAN_EXCLUSIVE_AREA_3	Using inside Tx confirmation to secure state of transmit object in case of cancellation (Only used when Vector Interface Version smaller 4.10 used). Or When Appl_GenericConfirmation() is used, lock against Can_Write() calls within (Data Consistency). > Duration is medium > Call to CanIf_CancelTxConfirmation() inside (no more calls in CanIf). > Disable global interrupts - or - Disable CAN interrupts and do not call function Can_Write() within.
CAN_EXCLUSIVE_AREA_4	Using inside received data handling (Rx Queue treatment) to secure Rx Queue counter and data. > Duration is short > No API call of other BSW inside. > Disable Global Interrupts - or - Disable all CAN interrupts.

CAN_EXCLUSIVE_AREA_5	Using inside wakeup handling to secure state transition. (Only in wakeup polling mode) Wakeup polling must not be interrupted by any mode change like triggered by Can_SetControllerMode(), BUSOFF-handling, Can_Init(), Can_ChangeBaudrate() or Can_SetBaudrate(). > Duration is short > Call to DET inside. > Use CAN interrupt locks here, in case the above mentioned APIs are only called within same task level or CAN interrupt context. or No action (locks) necessary when all APIs (mentioned above) called from same context or cannot interrupt each other. or Disable global interrupts
CAN_EXCLUSIVE_AREA_6	Using inside Can_SetControllerMode() and BUSOFF to avoid nested state transition requests. Can_SetControllerMode() and BUSOFF-handling (polling or interrupt) must not be interrupted by other calls of Can_SetControllerMode(), BUSOFF-handling, Can_Init(), Can_ChangeBaudrate() or Can_SetBaudrate() > Duration is medium > No API call of other BSW inside. > Use CAN interrupt locks here, in case the above mentioned APIs are only called within same task level or CAN interrupt context. or No action (locks) necessary when all APIs (mentioned above) called from same context or cannot interrupt each other. or Disable global interrupts
CAN_EXCLUSIVE_AREA_7	Using inside received data handling (Tx hardware FIFO treatment) to secure Tx hardware FIFO counter and data. > Duration is short > No API call of other BSW inside. > Disable Global Interrupts - or - Disable all CAN interrupts.

Table 5-3 Critical Section Codes

5.4 Compiler Abstraction and Memory Mapping

The objects (e.g. variables, functions, constants) are declared by compiler independent definitions – the compiler abstraction definitions. Each compiler abstraction definition is assigned to a memory section.

The following table contains the memory section names and the compiler abstraction definitions defined for the CAN Interface and illustrates their assignment among each other.

Memory Mapping Sections \ Compiler Abstraction Definitions	CAN_ CODE	CAN_STATIC_ CODE	CAN_ CONST	CAN_ CONST_PBCFG	CAN_VAR_NOINIT	CAN_ VAR_INIT	CAN_VAR_PBCFG	CAN_ INT_CTRL	CAN_ REG_CANCELL	CAN_ RX_TX_DATA	CAN_ APPL_CODE	CAN_ APPL_CONST	CAN_ APPL_VAR
CAN_START_SEC_CODE CAN_STOP_SEC_CODE	■												
CAN_START_SEC_STATIC_CODE CAN_STOP_SEC_STATIC_CODE		■											
CAN_START_SEC_CONST_8BIT CAN_STOP_SEC_CONST_8BIT			■										
CAN_START_SEC_CONST_16BIT CAN_STOP_SEC_CONST_16BIT			■										
CAN_START_SEC_CONST_32BIT CAN_STOP_SEC_CONST_32BIT			■										
CAN_START_SEC_CONST_UNSPECIFIED CAN_STOP_SEC_CONST_UNSPECIFIED			■										
CAN_START_SEC_PBCFG CAN_STOP_SEC_PBCFG				■									
CAN_START_SEC_PBCFG_ROOT CAN_STOP_SEC_PBCFG_ROOT				■									
CAN_START_SEC_VAR_NOINIT_UNSPECIFIED CAN_STOP_SEC_VAR_NOINIT_UNSPECIFIED					■								
CAN_START_SEC_VAR_INIT_UNSPECIFIED CAN_STOP_SEC_VAR_INIT_UNSPECIFIED						■							
CAN_START_SEC_VAR_PBCFG CAN_STOP_SEC_VAR_PBCFG							■						
CAN_START_SEC_CODE_APPL CAN_STOP_SEC_CODE_APPL											■		

Table 5-4 Compiler abstraction and memory mapping

The Compiler Abstraction Definitions CAN_ APPL_CODE, CAN_ APPL_VAR and CAN_ APPL_CONST are used to address code, variables and constants which are declared by other modules and used by the CAN Driver.

These definitions are not mapped by the CAN Driver but by the memory mapping realized in the CAN Interface or direct by application.

CAN_ CODE: used for CAN module code.

CAN_ STATIC_CODE: used for CAN module local code.

CAN_ CONST: used for CAN module constants.

CAN_ CONST_PBCFG: used for CAN module constants in Post-Build section.

CAN_ VAR_*: used for CAN module variables.

CAN_ INT_CTRL: is used to access the CAN interrupt controls.

CAN_ REG_CANCELL: is used to access the CAN Cell itself.

CAN_ RX_TX_DATA: access to CAN data buffers

CAN_ APPL_*: access to higher layers.

5.5 Hardware Specific Hints

BusName given by configuration have to be the same name as inside CANoe configuration for BusName, to map the logical controller to a specified CAN node.

6 API Description

6.1 Can_InitMemory

Prototype	
`void Can_InitMemory (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Power-up memory initialization. Initializes component variables in *_INIT_* sections at power up. Use this to re-run the system without performing a new start from power on. (E.g.: used to support an ongoing debug session without a complete re-initialization.)	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-1 Can_InitMemory

6.2 Can_Init

Prototype	
`void Can_Init (Can_ConfigPtrType ConfigPtr)`	
Parameter	
ConfigPtr [in]	Component configuration structure
Return code	
void	none
Functional Description	
Initializes component. Initializes all component variables and sets the component state to initialized.	
Particularities and Limitations	
> Interrupts are disabled. Module is uninitialized. Can_InitMemory() has been called unless CAN Module is initialized by start up code.	
Call context	

> TASK > This function is Synchronous > This function is Non-Reentrant

Table 6-2 Can_Init

6.3 Can_ChangeBaudrate

Prototype	
`Std_ReturnType` **`Can_ChangeBaudrate`** `(uint8 Controller, const uint16 Baudrate)`	
Parameter	
Controller [in]	CAN controller to be changed
Baudrate [in]	Baud rate to be set
Return code	
Std_ReturnType	E_NOT_OK Baud rate is not set
Std_ReturnType	E_OK Baud rate is set
Functional Description	
Change baud rate. This service shall change the baud rate and reinitialize the CAN controller.	
Particularities and Limitations	
The CAN controller must be in "STOP Mode". Configuration Variant(s): (CAN_CHANGE_BAUDRATE_API == STD_ON) || (CAN_SET_BAUDRATE_API == STD_OFF) Has to be called during the start up sequence before CAN communication takes place but after calling "Can_Init()".	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-3 Can_ChangeBaudrate

6.4 Can_CheckBaudrate

Prototype	
`Std_ReturnType` **`Can_CheckBaudrate`** `(uint8 Controller, const uint16 Baudrate)`	
Parameter	
Controller [in]	CAN controller to be checked
Baudrate [in]	Baud rate to be checked

Return code	
Std_ReturnType	E_NOT_OK Baud rate is not available
Std_ReturnType	E_OK Baud rate is available
Functional Description	
Checks baud rate. This service shall check if the given baud rate is supported of the CAN controller.	
Particularities and Limitations	
The CAN controller must be initialized. Configuration Variant(s): CAN_CHANGE_BAUDRATE_API == STD_ON	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-4 Can_CheckBaudrate

6.5 Can_SetBaudrate

Prototype	
`Std_ReturnType Can_SetBaudrate (uint8 Controller, uint16 BaudRateConfigID)`	
Parameter	
Controller [in]	CAN controller to be set
BaudRateConfigID [in]	Identity of the configured baud rate (available as Symbolic Name)
Return code	
Std_ReturnType	E_NOT_OK Baud rate is not set
Std_ReturnType	E_OK Baud rate is set
Functional Description	
Set baud rate. This service shall change the baud rate and reinitialize the CAN controller. (Similar to Can_ChangeBaudrate() but used when identical baud rates are used for different CAN FD settings).	
Particularities and Limitations	
The CAN controller must be in "STOP Mode". Configuration Variant(s): CAN_SET_BAUDRATE_API == STD_ON	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-5 Can_SetBaudrate

6.6 Can_GetVersionInfo

Prototype	
`void Can_GetVersionInfo (Can_VersionInfoPtrType VersionInfo)`	
Parameter	
VersionInfo [out]	Pointer to where to store the version information. Parameter must not be NULL.
Return code	
void	none
Functional Description	
Returns the version information. Returns version information (MICROSAR3: BCD coded), vendor ID and AUTOSAR module ID.	
Particularities and Limitations	
- Configuration Variant(s): CAN_VERSION_INFO_API == STD_ON	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 6-6 Can_GetVersionInfo

6.7 Can_GetStatus

Prototype	
`uint8 Can_GetStatus (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller requested for status information
Return code	
uint8	CAN_STATUS_START START mode
	CAN_STATUS_STOP STOP mode
	CAN_STATUS_INIT Initialized controller
	CAN_STATUS_INCONSISTENT STOP or SLEEP are inconsistent over common CAN controllers
	CAN_DEACTIVATE_CONTROLLER RAM check failed CAN controller is deactivated
	CAN_STATUS_WARNING WARNING state
	CAN_STATUS_PASSIVE PASSIVE state

	CAN_STATUS_BUSOFF BUSOFF mode
	CAN_STATUS_SLEEP SLEEP mode
Functional Description	
Get status/mode of the given controller. Delivers the status of the hardware, as a bit coded value where multiple bits may be set.	
Particularities and Limitations	
- Configuration Variant(s): CAN_GET_STATUS == STD_ON the return value can be analysed using the provided API macros: CAN_HW_IS_OK(), CAN_HW_IS_WARNING(), CAN_HW_IS_PASSIVE()CAN_HW_IS_BUSOFF(), CAN_HW_IS_WAKEUP(), CAN_HW_IS_SLEEP()CAN_HW_IS_STOP(), CAN_HW_IS_START(), CAN_HW_IS_INCONSISTENT() None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 6-7 Can_GetStatus

6.8 Can_GetControllerMode

Prototype	
`Std_ReturnType` **`Can_GetControllerMode`** `(uint8 Controller, Can_ControllerStatePtrType ControllerModePtr)`	
Parameter	
Controller [in]	CAN controller requested for status information
ControllerModePtr [out]	pointer to CAN controllers mode
Return code	
Std_ReturnType	E_NOT_OK Controller mode request has not been accepted
Std_ReturnType	E_OK Controller mode request has been accepted
Functional Description	
Get mode of the given controller. Delivers the controllers mode.	
Particularities and Limitations	
- Configuration Variant(s): CAN_GET_STATUS == STD_ON	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 6-8 Can_GetControllerMode

6.9 Can_GetControllerErrorState

Prototype	
`Std_ReturnType` **`Can_GetControllerErrorState`** `(uint8 Controller, Can_ErrorStatePtrType ErrorStatePtr)`	
Parameter	
Controller [in]	CAN controller requested for status information
ErrorStatePtr [out]	pointer to CAN controllers error state
Return code	
Std_ReturnType	E_NOT_OK Controller state request has not been accepted
Std_ReturnType	E_OK Controller state request has been accepted
Functional Description	
Get status of the given controller. Delivers the controllers error state.	
Particularities and Limitations	
- Configuration Variant(s): CAN_GET_STATUS == STD_ON	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 6-9 Can_GetControllerErrorState

6.10 Can_GetControllerTxErrorCounter

Prototype	
`Std_ReturnType` **`Can_GetControllerTxErrorCounter`** `(uint8 Controller, Can_ErrorCounterPtrType TxErrorCounterPtr)`	
Parameter	
Controller [in]	CAN controller requested for TX error counter
TxErrorCounterPtr [out]	pointer to CAN controllers TX error counter
Return code	
Std_ReturnType	E_NOT_OK Controller TX error counter not available
Std_ReturnType	E_OK Controller TX error counter available

Functional Description
Get status of the given controller. Delivers the controllers TX error counter.
Particularities and Limitations
- Configuration Variant(s): CAN_GET_STATUS == STD_ON
Call context
> ANY > This function is Synchronous > This function is Reentrant

Table 6-10 Can_GetControllerTxErrorCounter

6.11 Can_GetControllerRxErrorCounter

Prototype	
`Std_ReturnType` **`Can_GetControllerRxErrorCounter`** `(uint8 Controller, Can_ErrorCounterPtrType RxErrorCounterPtr)`	
Parameter	
Controller [in]	CAN controller requested for RX error counter
RxErrorCounterPtr [out]	pointer to CAN controllers RX error counter
Return code	
Std_ReturnType	E_NOT_OK Controller RX error counter not available
Std_ReturnType	E_OK Controller RX error counter available
Functional Description	
Get status of the given controller. Delivers the controllers RX error counter.	
Particularities and Limitations	
- Configuration Variant(s): CAN_GET_STATUS == STD_ON	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 6-11 Can_GetControllerRxErrorCounter

6.12 Can_SetControllerMode

Prototype	
`Can_ReturnType **Can_SetControllerMode** (uint8 Controller, Can_StateTransitionType Transition)`	
Parameter	
Controller [in]	CAN controller to be set
Transition [in]	Requested transition to destination mode CAN_T_START, CAN_T_STOP, CAN_T_SLEEP, CAN_T_WAKEUP
Return code	
Can_ReturnType	CAN_NOT_OK transition request rejected
Can_ReturnType	CAN_OK transition request accepted
Functional Description	
Change the controller mode. Request a mode transition that will be processed a short time synchronous, (may be finished ansynconous in Can_Mainfunction_Mode()) and call a notification when successful. BUSOFF, WAKEUP and RAM check will be handled as well.	
Particularities and Limitations	
Interrupts locked	
Call context	
> ANY > This function is Non-Reentrant	

Table 6-12 Can_SetControllerMode

6.13 Can_Write

Prototype	
`Can_ReturnType **Can_Write** (Can_HwHandleType Hth, Can_PduInfoPtrType PduInfo)`	
Parameter	
Hth [in]	Handle of the mailbox intended to send the message
PduInfo [in]	Information about the outgoing message (ID, dataLength, data)
Return code	
Can_ReturnType	CAN_NOT_OK transmit request rejected
	CAN_OK transmit request successful
	CAN_BUSY transmit request could not be accomplished due to the controller is busy.
Functional Description	
Send a CAN message. copy data, DLC and ID to the send mailbox and request a transmission.	

Particularities and Limitations
disabled CAN interrupts / interrupts locked (Refer to critical section).
Call context
> ANY > This function is Synchronous > This function is Non-Reentrant

Table 6-13 Can_Write

6.14 Can_CancelTx

Prototype	
`void` **`Can_CancelTx`** `(Can_HwHandleType Hth, PduIdType PduId)`	
Parameter	
Hth [in]	Handle of the mailbox intended to be cancelled.
PduId [in]	PDU identifier
Return code	
void	none
Functional Description	
Cancel TX message. Cancel the TX message in the hardware buffer (if possible) or mark the message as not to be confirmed in case of the cancellation is unsuccessful.	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-14 Can_CancelTx

6.15 Can_SetMirrorMode

Prototype	
`void` **`Can_SetMirrorMode`** `(uint8 Controller, CddMirror_MirrorModeType mirrorMode)`	
Parameter	
Controller [in]	CAN controller
mirrorMode [in]	Activate or deactivate the mirror mode.

Return code	
void	none
Functional Description	
Activate mirror mode. Switch the Appl_GenericPreCopy/Confirmation function ON or OFF.	
Particularities and Limitations	
- Configuration Variant(s): C_ENABLE_MIRROR_MODE (user configuration file) Called by "Mirror Mode" CDD. None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-15 Can_SetMirrorMode

6.16 Can_SetSilentMode

Prototype	
`Std_ReturnType Can_SetSilentMode (uint8 Controller, Can_SilentModeType silentMode)`	
Parameter	
Controller [in]	CAN controller
silentMode [in]	Activate or deactivate the silent mode CAN_SILENT_INACTIVE, CAN_SILENT_ACTIVE (Enumaration).
Return code	
Std_ReturnType	E_OK mode change successful
Std_ReturnType	CAN_OK mode change failed
Functional Description	
Activate and deactivate the silent mode. Switch to silent mode, as a listen only mode without ACK. Also deactivate this mode again.	
Particularities and Limitations	
The CAN controller must be in "STOP Mode". Configuration Variant(s): CAN_SILENT_MODE == STD_ON	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-16 Can_SetSilentMode

6.17 Can_CheckWakeup

Prototype	
`Can_ReturnType` **`Can_CheckWakeup`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to be checked for WAKEUP events.
Return code	
Can_ReturnType	E_OK/CAN_OK the given controller caused a WAKEUP before.
Can_ReturnType	E_NOT_OK/CAN_NOT_OK the given controller caused no WAKEUP before.
Functional Description	
Check WAKEUP occur. Check the occurrence of WAKEUP events for the given controller (used as WAKEUP callback for higher layers).	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-17 Can_CheckWakeup

6.18 Can_DisableControllerInterrupts

Prototype	
`void` **`Can_DisableControllerInterrupts`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to disable interrupts for.
Return code	
void	none
Functional Description	
Disable CAN interrupts. Disable the CAN interrupt for the given controller (e.g. due to data consistency reasons).	
Particularities and Limitations	
Must not be called while CAN controller is in SLEEP mode.	
Call context	
> ANY	

> This function is Synchronous > This function is Non-Reentrant

Table 6-18 Can_DisableControllerInterrupts

6.19 Can_EnableControllerInterrupts

Prototype	
`void Can_EnableControllerInterrupts (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to enable interrupts for.
Return code	
void	none
Functional Description	
Enable CAN interrupts. Re-enable the CAN interrupt for the given controller (e.g. due to data consistency reasons).	
Particularities and Limitations	
Must not be called while CAN controller is in SLEEP mode.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-19 Can_EnableControllerInterrupts

6.20 Can_MainFunction_Write

Prototype	
`void Can_MainFunction_Write (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
TX message observation. Polling TX events (confirmation, cancellation) for all controllers and all TX mailboxes to accomplish the TX confirmation handling (like CanInterface notification).	

Particularities and Limitations
Must not interrupt the call of Can_Write(). CAN driver has to be initialized.
Call context
> TASK > This function is Synchronous > This function is Non-Reentrant

Table 6-20 Can_MainFunction_Write

6.21 Can_MainFunction_Read

Prototype	
`void` **`Can_MainFunction_Read`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
RX message observation. Polling RX events for all controllers and all RX mailboxes to accomplish the RX indication handling (like CanInterface notification). Also used for a delayed read (from task level) of the RX Queue messages which were queued from interrupt context.	
Particularities and Limitations	
CAN driver has to be initialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-21 Can_MainFunction_Read

6.22 Can_MainFunction_BusOff

Prototype	
`void` **`Can_MainFunction_BusOff`** `(void)`	
Parameter	
void	none

Return code	
void	none
Functional Description	
BUSOFF observation. Polling of BUSOFF events to accomplish the BUSOFF handling. Service function to poll BUSOFF events for all controllers to accomplish the BUSOFF handling (like calling of CanIf_ControllerBusOff() in case of BUSOFF occurrence).	
Particularities and Limitations	
CAN driver has to be initialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-22 Can_MainFunction_BusOff

6.23 Can_MainFunction_Wakeup

Prototype	
`void Can_MainFunction_Wakeup (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
WAKEUP observation. Polling WAKEUP events for all controllers to accomplish the WAKEUP handling (like calling of "CanIf_SetWakeupEvent()" in case of WAKEUP occurrence).	
Particularities and Limitations	
CAN driver has to be initialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-23 Can_MainFunction_Wakeup

6.24 Can_MainFunction_Mode

Prototype	
`void` **`Can_MainFunction_Mode`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Mode transition observation. Polling Mode changes over all controllers. (This is handled asynchronous if not accomplished in "Can_SetControllerMode()")	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-24 Can_MainFunction_Mode

6.25 Can_RamCheckExecute

Prototype	
`void` **`Can_RamCheckExecute`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to be checked.
Return code	
void	none
Functional Description	
Start checking the CAN cells RAM. Check all controller specific and mailbox specific registers by write patterns and read back. Issue notification will appear in this context.	
Particularities and Limitations	
Has to be called within STOP mode. Configuration Variant(s): CAN_RAM_CHECK == CAN_EXTENDED	
Call context	
> TASK > This function is Synchronous	

> This function is Non-Reentrant

Table 6-25 Can_RamCheckExecute

6.26 Can_RamCheckEnableMailbox

Prototype	
`void Can_RamCheckEnableMailbox (Can_HwHandleType htrh)`	
Parameter	
htrh [in]	CAN mailbox to be reactivated.
Return code	
void	none
Functional Description	
Reactivate a mailbox after RamCheck failed. Mailbox will be reactivated by clear deactivation flag.	
Particularities and Limitations	
Has to be called within STOP mode after RamCheck failed (controller is deactivated). Configuration Variant(s): CAN_RAM_CHECK == CAN_EXTENDED	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 6-26 Can_RamCheckEnableMailbox

6.27 Can_RamCheckEnableController

Prototype	
`void Can_RamCheckEnableController (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to be reactivated.
Return code	
void	none
Functional Description	
Reactivate CAN cells after RamCheck failed. CAN cell will be reactivated by execute reinitialization.	
Particularities and Limitations	
Has to be called within STOP mode after RamCheck failed (controller is deactivated).	

Configuration Variant(s): CAN_RAM_CHECK == CAN_EXTENDED
Call context
> TASK > This function is Synchronous > This function is Non-Reentrant

Table 6-27 Can_RamCheckEnableController

6.28 CanSetHighVoltageMode

Prototype	
`void CanSetHighVoltageMode (CanChannelHandle Controller, vuint8 mode)`	
Parameter	
canChannel [in]	CAN controller
mode [in]	transceiver mode: kCanVoltageHigh (1) high voltage kCanVoltageNormal (0) normal
Return code	
void	none
Functional Description	
Set high voltage mode. use for a Single Wire CAN Transceiver (AU 5790, AU 5790/2, SWC 9832).	
Particularities and Limitations	
- Configuration Variant(s): -	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-28 CanSetHighVoltageMode

6.29 Can_RxIndication

Prototype	
`void Can_RxIndication (vuint8 canChannel, vuint32 id, vuint32 flags, vuint8 dataLength, const vuint8 *data)`	
Parameter	
canChannel [in]	CAN controller
id [in]	ID of the received message

dataLength [in]	data length of the received message
data [in]	pointer to the received message data
Return code	
void	none
Functional Description	
RX indication handling. Receiving message data from CANoe by copy received data into a emulated CAN cell buffer.	
Particularities and Limitations	
- Configuration Variant(s): -	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-29 Can_RxIndication

6.30 Can_TxConfirmation

Prototype	
`void Can_TxConfirmation (vuint8 canChannel, vuint32 id, vuint32 flags, vuint8 dataLength, const vuint8 *data)`	
Parameter	
channel [in]	CAN controller
id [in]	ID of the transmitted message
Return code	
void	none
Functional Description	
TX confirmation handling. handle TX confirmation from CANoe by mark emulated CAN cell mailbox as be confirmed.	
Particularities and Limitations	
- Configuration Variant(s): !defined(CAN_VTT_DRIVER)	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-30 Can_TxConfirmation

6.31 Can_ErrIndication

Prototype	
`void Can_ErrIndication (vuint8 canChannel, vuint32 errCode, vuint8 errCntTx, vuint8 errCntRx)`	
Parameter	
channel [in]	CAN controller
errCode [in]	error code specify the event
errCntTx [in]	TX error counter value
errCntRx [in]	RX error counter value
Return code	
void	none
Functional Description	
error handling handle the BUSOFF indication from CANoe	
Particularities and Limitations	
-	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-31 Can_ErrIndication

6.32 CanInterruptRx

Prototype	
`void CanInterruptRx (vuint8 canChannel)`	
Parameter	
Controller [in]	CAN controller to be checked for WAKEUP events.
Return code	
void	none
Functional Description	
RX ISR. handle the RX event. Search for mailbox that cause this event and call further RX handling.	
Particularities and Limitations	
-	

Call context
> ISR1 > This function is Synchronous > This function is Non-Reentrant

Table 6-32 CanInterruptRx

6.33 CanInterruptTx

Prototype	
`void CanInterruptTx (vuint8 canChannel)`	
Parameter	
Controller [in]	CAN controller to be checked for WAKEUP events.
Return code	
void	none
Functional Description	
TX ISR. handle the TX event. Search for mailbox that cause this event and call further TX handling.	
Particularities and Limitations	
-	
Call context	
> ISR1 > This function is Synchronous > This function is Non-Reentrant	

Table 6-33 CanInterruptTx

6.34 CanInterruptErr

Prototype	
`void CanInterruptErr (vuint8 canChannel)`	
Parameter	
Controller [in]	CAN controller to be checked for WAKEUP events.
Return code	
void	none
Functional Description	
Error ISR. set error id in emulated CAN cell and call further Error handling for BUSOFF.	

Particularities and Limitations
-
Call context
> ISR1 > This function is Synchronous > This function is Non-Reentrant

Table 6-34 CanInterruptErr

6.35 CanInterruptWakeup

Prototype	
`void CanInterruptWakeup (vuint8 canChannel)`	
Parameter	
Controller [in]	CAN controller to be checked for WAKEUP events.
Return code	
void	none
Functional Description	
WAKEUP ISR. check WAKEUP occur in SLEEP and call further WAKEUP handling.	
Particularities and Limitations	
-	
Call context	
> ISR1 > This function is Synchronous > This function is Non-Reentrant	

Table 6-35 CanInterruptWakeup

6.36 Appl_GenericPrecopy

Prototype	
`Can_ReturnType Appl_GenericPrecopy (uint8 Controller, Can_IdType ID, uint8 DataLength, Can_DataPtrType DataPtr)`	
Parameter	
Controller [in]	CAN controller which received the message.
ID [in]	ID of the received message (include IDE,FD). In case of extended or mixed ID systems the highest bit (bit 31) is set to mark an extended ID.

	FD-bit (bit 30) can be masked out with a define CAN_ID_MASK_IN_GENERIC_CALLOUT in the user config file.
DataLength [in]	Data length of the received message.
pData [in]	Pointer to the data of the received message (read only).
Return code	
Can_ReturnType	CAN_OK Higher layer indication will be called afterwards (CanIf_RxIndication()).
Can_ReturnType	CAN_NOT_OK Higher layer indication will not be called afterwards.
Functional Description	
Common RX indication callback that will be called before message specific callback will be called. Application callback function which informs about all incoming RX messages including the contained data. It can be used to block notification to upper layer. E.g. to filter incoming messages or route it for special handling.	
Particularities and Limitations	
- Configuration Variant(s): CAN_GENERIC_PRECOPY == STD_ON "pData" is read only and must not be accessed for further write operations. The parameter DataLength refers to the received data length by the CAN controller hardware. Note, that the CAN protocol allows the usage of data length values greater than eight (CAN-FD). Depending on the implementation of this callback it may be necessary to consider this special case (e.g. if the data length is used as index value in a buffer write access). None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-36 Appl_GenericPrecopy

6.37 Appl_GenericConfirmation

Prototype	
`Can_ReturnType` **`Appl_GenericConfirmation`** `(PduIdType PduId)`	
Parameter	
PduId [in]	Handle of the PDU specifying the message.
Return code	
Can_ReturnType	CAN_OK Higher layer confirmation will be called afterwards (CanIf_TxConfirmation()).
Can_ReturnType	CAN_NOT_OK Higher layer confirmation will not be called afterwards.
Functional Description	
Common TX notification callback that will be called before message specific callback will be called.	

Application callback function which informs about TX messages being sent to the CAN bus. It can be used to block confirmation or route the information to other layer as well.
Particularities and Limitations
- Configuration Variant(s): CAN_GENERIC_CONFIRMATION == STD_ON. "PduId" is read only and must not be accessed for further write operations. None AUTOSAR API
Call context
> ANY > This function is Synchronous > This function is Non-Reentrant

Table 6-37 Appl_GenericConfirmation

6.38 Appl_GenericConfirmation

Prototype	
`Can_ReturnType` **`Appl_GenericConfirmation`** `(uint8 Controller, Can_PduInfoPtrType DataPtr)`	
Parameter	
Controller [in]	CAN controller which send the message.
DataPtr [in]	Pointer to a Can_PduType structure including ID (include IDE,FD), DataLength, PDU and data pointer.
Return code	
Can_ReturnType	CAN_OK Higher layer (CanInterface) confirmation will be called.
Can_ReturnType	CAN_NOT_OK No further higher layer (CanInterface) confirmation will be called.
Functional Description	
Common TX notification callback that will be called before message specific callback will be called. Application callback function which informs about TX messages being sent to the CAN bus. It can be used to block confirmation or route the information to other layer as well.	
Particularities and Limitations	
- Configuration Variant(s): CAN_GENERIC_CONFIRMATION == CAN_API2. A new transmission within this call out will corrupt the DataPtr context. None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-38 Appl_GenericConfirmation

6.39 Appl_GenericPreTransmit

Prototype	
`void Appl_GenericPreTransmit (uint8 Controller, Can_PduInfoPtrType_var DataPtr)`	
Parameter	
Controller [in]	CAN controller on which the message will be send.
DataPtr [in]	Pointer to a Can_PduType structure including ID (include IDE,FD), DataLength, PDU and data pointer.
Return code	
void	none
Functional Description	
Common transmit callback. Application callback function allowing the modification of the data to be transmitted (e.g.: add CRC).	
Particularities and Limitations	
- Configuration Variant(s): CAN_GENERIC_PRETRANSMIT == STD_ON. None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-39 Appl_GenericPreTransmit

6.40 ApplCanTimerStart

Prototype	
`void ApplCanTimerStart (CanChannelHandle Controller, uint8 source)`	
Parameter	
Controller [in]	CAN controller on which the hardware observation takes place. (only if not using "Optimize for one controller")
source [in]	Source for the hardware observation.
Return code	
void	none
Functional Description	
Start time out monitoring. Service function to start an observation timer.	

Particularities and Limitations
-
Configuration Variant(s): (CAN_HW_LOOP_SUPPORT_API == STD_ON) && (CAN_HW_LOOP_SUPPORT_API == STD_ON). Please refer to chapter "Hardware Loop Check".
None AUTOSAR API
Call context
> ANY > This function is Synchronous > This function is Non-Reentrant

Table 6-40 ApplCanTimerStart

6.41 ApplCanTimerLoop

Prototype	
`Can_ReturnType` **`ApplCanTimerLoop`** `(CanChannelHandle Controller, uint8 source)`	
Parameter	
Controller [in]	CAN controller on which the hardware observation takes place. (only if not using "Optimize for one controller")
source [in]	Source for the hardware observation.
Return code	
Can_ReturnType	> CAN_NOT_OK when loop shall be broken (observation stops) > CAN_NOT_OK should only be used in case of a time out occurs due to a hardware issue. > After this an appropriate error handling is needed (see chapter Hardware Loop Check / Time out Monitoring).
Can_ReturnType	> CAN_OK when loop shall be continued (observation continues)
Functional Description	
Time out monitoring. Service function to check (against generated maximum loop value) whether a hardware loop shall be continued or broken.	
Particularities and Limitations	
-	
Configuration Variant(s): (CAN_HW_LOOP_SUPPORT_API == STD_ON) && (CAN_HW_LOOP_SUPPORT_API == STD_ON). Please refer to chapter "Hardware Loop Check".	
None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-41 ApplCanTimerLoop

6.42 ApplCanTimerEnd

Prototype	
`void ApplCanTimerEnd (CanChannelHandle Controller, uint8 source)`	
Parameter	
Controller [in]	CAN controller on which the hardware observation takes place. (only if not using "Optimize for one controller")
source [in]	Source for the hardware observation.
Return code	
void	none
Functional Description	
Stop time out monitoring. Service function to to end an observation timer.	
Particularities and Limitations	
- Configuration Variant(s): (CAN_HW_LOOP_SUPPORT_API == STD_ON) && (CAN_HW_LOOP_SUPPORT_API == STD_ON). Please refer to chapter "Hardware Loop Check". None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-42 ApplCanTimerEnd

6.43 Appl_CanOverrun

Prototype	
`void Appl_CanOverrun (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller for which the overrun was detected.
Return code	
void	none
Functional Description	
Overrun detection. Called when an overrun is detected for a BasicCAN mailbox. Alternatively a DET call can be selected instead of ("CanOverrunNotification" is set to "DET").	

Particularities and Limitations
-
Configuration Variant(s): CAN_OVERRUN_NOTIFICATION == CAN_APPL. None AUTOSAR API
Call context
> ANY
> This function is Synchronous
> This function is Non-Reentrant

Table 6-43 Appl_CanOverrun

6.44 Appl_CanFullCanOverrun

Prototype	
`void` **`Appl_CanFullCanOverrun`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller for which the overrun was detected.
Return code	
void	none
Functional Description	
Overrun detection. Called when an overrun is detected for a FullCAN mailbox. Alternatively a DET call can be selected instead of ("CanOverrunNotification" is set to "DET").	
Particularities and Limitations	
- Configuration Variant(s): CAN_OVERRUN_NOTIFICATION == CAN_APPL. None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 6-44 Appl_CanFullCanOverrun

6.45 Services used by CAN

In the following table services provided by other components, which are used by the CAN are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	**Det_ReportError** (see “Development Error Reporting”)

Component	API
EcuM	**EcuM_CheckWakeup** This function is called when WAKEUP over CAN bus occur. **EcuM_GeneratorCompatibilityError** This function is called during the initialization, of the CAN Driver if the Generator Version Check or the CRC Check fails. (see [5])
Application (optional non AUTOSAR)	**Appl_GenericPrecopy** **Appl_GenericConfirmation** **Appl_GenericPreTransmit** **ApplCanTimerStart/Loop/End** **Appl_CanRamCheckFailed, Appl_CanCorruptMailbox** **ApplCanInterruptDisable/Restore** **Appl_CanOverrun, Appl_CanFullCanOverrun** For detailed description see Chapter 6.2
CANIF	**CanIf_CancelTxNotification** (non AUTOSAR) A special Software cancellation callback only used within Vector CAN Driver CAN Interface bundle. **CanIf_TxConfirmation** Notification for a successful transmission. (see [4]) **CanIf_CancelTxConfirmation** Notification for a successful Tx cancellation. (see [4]) **CanIf_RxIndication** Notification for a message reception. (see [4]) **CanIf_ControllerBusOff** BUSOFF notification function. (see [4]) **CanIf_ControllerModeIndication** Notification for mode successfully changed. **CanIf_ControllerErrorStatePassive** Notification for transition to error state passive (see [4]) **CanIf_ErrorNotification** Notification of the detection of a CanErrorType [4] (see [4]) Additional services used to support refer to chapter“Extended RAM Check”
Os	**OS_TICKS2MS_<counterShortName>()** Os macro to get timebased ticks from counter. **GetElapsedValue** Get elapsed tick count. **GetCounterValue** Get tick count start.

Table 6-45 Services used by the CAN

7 Configuration

The CAN Driver supports pre-compile and post-build configuration.

For post-build loadable systems, re-flashing the generated data can change some configuration settings.

For post-build and link-time configurations pre-compile settings are configured at compile time and therefore unchangeable at link or post-build time.

7.1 Pre-Compile Parameters

Some settings have to be available before compilation:

- Version API (Can_GetVersionInfo() activation)
  #define CAN_VERSION_INFO_API STD_ON/STD_OFF
- DET (development error detection)
  #define CAN_DEV_ERROR_DETECT STD_ON/STD_OFF
- Hardware Loop Check (timeout monitoring)
  #define CAN_HARDWARE_CANCELLATION STD_ON/STD_OFF
- Polling modes: Tx Confirmation, Rx Reception, `WAKEUP`, `BUSOFF`
  #define CAN_TX_PROCESSING CAN_INTERRUPT/ CAN_POLLING
  #define CAN_RX_PROCESSING CAN_INTERRUPT/ CAN_POLLING
  #define CAN_BUSOFF_PROCESSING CAN_INTERRUPT/ CAN_POLLING
  #define CAN_WAKEUP_PROCESSING CAN_INTERRUPT/ CAN_POLLING
  #define CAN_INDIVIDUAL_PROCESSING STD_ON/STD_OFF
- Multiplexed Tx (external PIA – by use multiple Tx mailboxes)
  #define CAN_MULTIPLEXED_TRANSMISSION STD_ON/STD_OFF
  (global switch see above + for each BasicCANTx mailbox configurable)
- Tx Hardware FIFO
  #define CAN_TX_HW_FIFO STD_ON/STD_OFF
  (global switch see above + for each BasicCANTx mailbox configurable)
- Configuration Variant (define the configuration type when using post-build variant)
  #define CAN_ENABLE_SELECTABLE_PB
- Use Generic Precopy Function (None AUTOSAR feature)
  #define CAN_GENERIC_PRECOPY STD_ON/STD_OFF
- Use Generic Confirmation Function (None AUTOSAR feature)
  #define CAN_GENERIC_CONFIRMATION STD_ON/STD_OFF
- Use Rx Queue Function (None AUTOSAR feature)
  #define CAN_RX_QUEUE STD_ON/STD_OFF
- Used ID type (standard/extended or mixed ID format)
  #define CAN_EXTENDED_ID STD_ON/STD_OFF
  #define CAN_MIXED_ID STD_ON/STD_OFF

- Usage of Rx and Tx Full and BasicCAN objects (deactivate only when not using and to save ROM and runtime consumption)
  #define CAN_RX_FULLCAN_OBJECTS STD_ON/STD_OFF
  #define CAN_TX_FULLCAN_OBJECTS STD_ON/STD_OFF
  #define CAN_RX_BASICCAN_OBJECTS STD_ON/STD_OFF
- Use Multiple BasicCAN objects
  #define CAN_MULTIPLE_BASICCAN STD_ON/STD_OFF
- Use Multiple BasicCAN Tx objects
  #define CAN_MULTIPLE_BASICCAN_TX STD_ON/STD_OFF
- Optimizations
  #define CAN_ONE_CONTROLLER_OPTIMIZATION STD_ON/STD_OFF
  #define CAN_DYNAMIC_FULLCAN_ID STD_ON/STD_OFF
- Usage of nested CAN interrupts
  #define CAN_NESTED_INTERRUPTS STD_ON/STD_OFF
- Use Multiple ECU configurations
  #define CAN_MULTI_ECU_CONFIG STD_ON/STD_OFF
- Use RAM Check (verify mailbox buffers)
  #define CAN_RAM_CHECK x
- Use Overrun detection
  #define CAN_OVERRUN_NOTIFICATION x
- Tx Cancellation
  #define CAN_HW_TRANSMIT_CANCELLATION STD_ON/STD_OFF
- Tx Cancellation of Identical IDs
  #define CAN_IDENTICAL_ID_CANCELLATION STD_ON/STD_OFF

7.2 Link-Time Parameters

The library version of the CAN Driver use following generated settings:

- Rx Queue size
- CAN Controller mapping (mapping of logical channel to CAN Controller).
- CAN Controller specific Wakeup support.

7.3 Post-Build Parameters

Following settings are post-build data that can be changed for re-flashing:

- Amount and usage of FullCAN Rx and Tx mailboxes
- Used database (message information like ID, DLC)
- Filters for BasicCAN Rx mailbox
- Baudrate settings

- Module start address (only for post-build loadable systems: The memory location for re-flashed data has to be defined)
- Configuration ID (only for post-build loadable systems: This number is used to identify the post-build data

7.4 Configuration with da DaVinci Configurator

See Online help within DaVinci Configurator and BSWMD file for parameter settings.

8 Glossary and Abbreviations

8.1 Glossary

Term	Description
High End (license)	Product license to support an extended feature set (see Table 4-1 Supported features)

Table 8-1 Glossary

8.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR / ASR	Automotive Open System Architecture
BSW	Basis Software
DET	Development Error Tracer
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MICROSAR / MSR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
SWS	Software Specification
Hardware Loop Check	Timeout monitoring for possible endless loops.
`START`	Start Mode according to AUTOSAR Specification[1] (see 4.4.1)
`STOP`	Stop Mode according to AUTOSAR Specification[1] (see 4.4.2)
`SLEEP`	Sleep Mode according to AUTOSAR Specification[1] (see 4.4.3)
`WAKEUP`	Wakeup Transition according to AUTOSAR Specification[1]
`BUSOFF`	Bus Off Transition according to CAN Driver Specification[6] (see 4.4.4)
`OVERRUN`	New, none processed messages will be lost, see 4.8.1.2 for more details
`OVERWRITE`	Old, none processed messages will be lost, see 4.8.1.2 for more details

Table 8-2 Abbreviations

9 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com